**CS1910 Computer Science I • Problem Solving and Debugging** **Try-at-home workbook**

# Bit Parity

On your own, at your own pace • about 60 minutes

**Name:** _____ **Partner:** _____

*This PDF can be filled in: click in an answer space to type, or print it and write. Draw diagrams with your PDF viewer's pen or markup tool, or on paper.*

## What this workbook practises

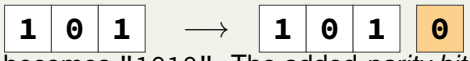
A *bit* has two states, which we write as `0` and `1`. Computers add a *parity bit* to data so that the receiver can detect some transmission errors. You will work through all four stages of problem solving, **Understand** → **Design** → **Implement** → **Evaluate**: pinning the problem down, breaking it into small reusable functions, writing them, and testing them, including finding out what kinds of error parity can and cannot catch. The Colab notebook `CS1910_Problem-Solving-Debugging-Parity_Handout.ipynb` has the same activity with automatic checks.

## 1 Understand

about 15 minutes

### The problem, as given

A bit string is a string containing only `0`s and `1`s, for example `"0100110"`. If it contains an even number of `1`s, we say it has **even parity**. Suppose we always want our bit strings to have even parity. Given a bit string, add one character to the end so that the new bit string has even parity.




Example: `"101"` becomes `"1010"`. The added *parity bit* is shown in gold.

**1.** What is the input, and what is its type? What is the output, and what is its type?

**2.** Is the output the bit that is added, or the whole new string? How can you tell?

**3.** What should happen for the empty string `""`? Explain.

4. What should happen if the input contains something other than `0` and `1`, such as `"102"`? Does the problem say?

5. How could the problem be generalized?

## Work examples by hand

6. Count the `1`s in each bit string, then write the parity bit and the whole new string.

| Bit string | Number of 1s | Parity bit | New string |
|---|---|---|---|
| 1 0 1 | 2 | 0 | `"1010"` |
| 1 1 1 | | | |
| 0 1 0 0 1 1 0 | | | |
| 0 0 0 0 | | | |
| "" (empty) | | | |

## Predict the output

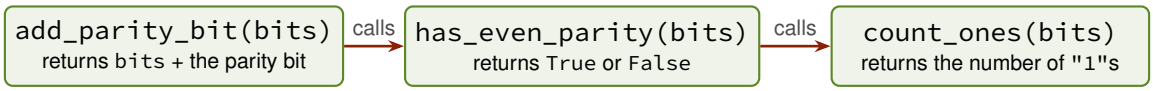
7. Write what each line prints. If it causes an error, name the error. These are the building blocks you will need.

| Code | Output |
|---|---|
| `print("101" + "0")` | |
| `print("101" + 0)` | |
| `print(3 % 2, 4 % 2)` | |
| `print(str(3 % 2))` | |
| `print("0100110".count("1"))` | |

## 2 Design

about 10 minutes

**Decompose** the problem into small functions that each do one job, and can be reused:




8. **Pattern recognition:** look at your table of examples. What is the connection between the number of 1s and the parity bit?

9. Write pseudocode for each of the three functions.

## 3 Implement

about 15 minutes

**10.** Write the three functions in Python. Write `count_ones` with a **loop** (not the `count` method), and **reuse** your functions: `has_even_parity` calls `count_ones`, and `add_parity_bit` calls `has_even_parity`.

## 4 Evaluate

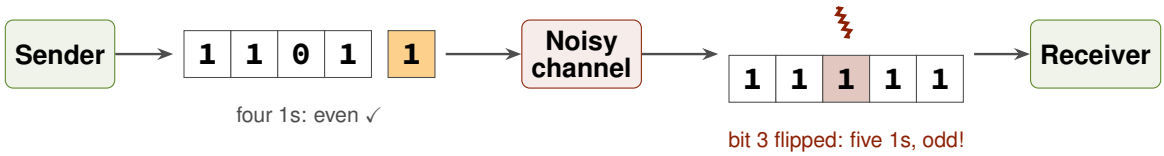
about 15 minutes

**11.** Before you trust your code, list your test cases. Include normal and edge cases, and work out each expected output first.

| Input | Expected output | Normal or edge? Why? |
|---|---|---|
| | | |
| | | |
| | | |
| | | |
| | | |

## Why parity? Detecting errors

The sender adds a parity bit, then sends the message. The receiver checks that what arrived still has even parity. On the way, noise can *flip* a bit: a `0` becomes a `1`, or the other way round.




**12.** Explain why the receiver can always detect a **single** flipped bit, even if the flipped bit is the parity bit itself.

**13.** The message `10001` (two `1`s, even) is sent, but bits 2 and 4 are flipped on the way, so this arrives: 1 1 0 1 1. Does the receiver notice? Explain.

**14.** When the receiver detects an error, can it tell *which* bit was wrong? What could it do?

**15.** Write `flip_bit(bits, i)`, which returns `bits` with the character at index `i` flipped. *Hint:* slicing, because strings are immutable.

**Extensions**

⋆ Generalize: add a parameter `even=True` to `add_parity_bit`; when `even` is `False`, add the bit that gives *odd* parity. What does `add_parity_bit("101", even=False)` return? _____

⋆⋆ Robustness: write `is_bit_string(text)`, which returns `True` only if every character is `"0"` or `"1"`. What should it return for `""`? _____

## ✓ Before you move on

about 5 minutes

Tick each statement you can honestly agree with.

- ☐ I can explain what even parity is, and add a parity bit to any bit string, including the empty string.
- ☐ I broke the problem into small functions and reused them.
- ☐ I chose normal and edge test cases before trusting my code.
- ☐ I can explain which errors a parity bit detects and which it misses.

**16.** Which stage of the process took you longest? Was it the one you expected?

**17.** Where else might a computer check data for errors?